

A bias study and an unbiased deep neural network for recommender systems

Li He^a, Jiashu Zhao^{b,*}, Yulong Gu^c, Mitchell Elbaz^b and Zhuoye Ding^a

^a JD.com, China

E-mails: 1632226712@qq.com, dingzhuoye@jd.com

^b Wilfrid Laurier University, Canada

E-mails: jzhao@wlu.ca, elba3790@mylaurier.ca

^c Bytedance, China

E-mail: guyulongcs@gmail.com

Abstract. User feedback data (e.g., clicks, dwell time in the product detail page) have been incorporated in the training process of many ranking models for better performance. Such approaches are widely used in many ranking applications, including search and recommendation. Recently, the inherent biases in user feedback data have been studied, which indicates how the users' behaviors can be affected by factors other than relevancy. By identifying and removing these biases, the ranking models can be further improved. Researchers have developed a variety of debiasing methods on different bias factors. Most of them only focus on one type of bias and pay little attention to different types of bias from a unified perspective. In this paper, we conduct a comprehensive study of bias focusing on the application of ranking problems in recommender systems which is highly important for the research of web intelligence. Then, we share our experiences derived from designing and optimizing unbiased models to improve feeds recommendation. To uncover the effects of biases and achieve better ranking performance, we propose several unbiased models and compare with state-of-the-art models. We conduct extensive offline experiments on real datasets and validate the effectiveness of our method by performing online A/B testing in a real-world recommender system.

Keywords: Bias study, unbiased learning to rank, recommender systems, neural network

1. Introduction

Users have many behaviors on a website/app, such as clicks, dwell time, purchase, etc. These user feedback data are valuable for machine learning models to explore the underlying patterns, and therefore estimate better ranking results. For example, for a given query, a clicked document is regarded as relevant and a non-clicked document is regarded as non-relevant. Then these labelled data can be utilized to train a classification model [15,52,54]. Using these implicit feedback data has many advantages: (1) Rich information is implied, e.g. users' interestingness and satisfaction level; (2) The training data is easy to collect and do not require high-cost manual labelling; (3) The feedback data is user-centric and therefore the ranked results tend to be more attractive to users. However, such user behaviors have inherent biases [4,5,16,30–32,38,46,51], i.e., the users can be affected/distracted by other factors than relevancy. For example, the higher ranked items are more likely to be clicked, which is the so-called *position bias*. In this case, using click data as a positive signal directly in learning to rank methods can lead to sub-optimal results and would cause a feedback loop effect.

*Corresponding author. E-mail: jzhao@wlu.ca.

In literature, researchers have proposed many debiasing methods on different bias factors. Nevertheless, most of works only focus on one type of bias and pay little attention to different types of bias from a unified point of view. Motivated by this issue, in this paper, we conduct a comprehensive study of bias to identify the appropriate bias factors in learning to rank and discuss the approaches to remove them. First, we formally give a definition of bias. Specifically, bias refers to those factors that will affect customer's click behaviors, which are only available during offline training, but cannot be obtained or is inconvenient to be obtained in online inference. Next, we identify three types of bias factors, including position bias factor, items' context bias factor, and users' status bias factor. Then specifically, we introduce some concrete biases in e-commerce feeds recommendation. Finally, we give the bias estimation methods and introduce how to debias those biases in learning ranking model.

After a comprehensive study of bias, we share our experiences in designing and optimizing models to improve feeds recommendation with bias. Modern e-commerce sites (e.g. Amazon, Alibaba) offer hundreds of millions of products for sale, which is challenging for customers to find their interested items. With the emergence of smart-phones, product recommendation has been widely used in mobile websites and apps, which provides users an interactive manner of blended results of recommended products in never-ending feeds. Specifically, when interacting with product stream, the users could click on the items and enter the product detail page. Meanwhile, one could also skip unattractive items and scroll down. The abundant implicit user feedbacks (e.g., clicks, dwell time in the product detail page) are collected for training ranking models. In E-commerce application, the following concepts will be used interchangeably: item/product, user/customer and feed/product recommendation.

To uncover the effects of biases and achieve better ranking performance, we propose several unbiased models by leveraging bias embeddings in deep neural networks [23,56]. We build this work in a real-world e-commerce portal and the proposed idea and approaches can be applied to other learning to rank systems. Empirically, we conduct extensive offline experiments on datasets gathered from a real-world e-commerce portal and validate the effectiveness of our method by performing online A/B testing in a real-world e-commerce recommender system. Experimental results demonstrate that our unbiased models is able to achieve superior quality of recommendation against state-of-the-art baselines.

In summary, we highlight our contributions as follows:

- To the bias our knowledge, we are the first to conduct a comprehensive study of bias, including formally proposing a bias definition and discuss how to distinguish factors as bias or not.
- We identify three unified types of biases, as well as design the bias estimation for unbiased learning to rank models.
- We propose several unbiased neural network models by adopting deep neural networks and bias embedding to reduce the effect of bias.
- The proposed models are instantiated on a real-world E-commerce recommendation problem which is very important for Web Intelligence research, and can achieve better performance compared to several state-of-the-art models.

The rest of this paper is organized as follows. Section 2 discusses related works about ranking models, click models, unbiased learning algorithms, and bias in recommender system. Section 3 gives a comprehensive study of bias, including the definition of bias, three types of bias, and example biases in recommender system. We share our model design experience of an unbiased model in Section 4. Our experimental setups and results are described in Section 5. Finally, we conclude this paper in Section 6.

2. Related work

2.1. Ranking models

The ranking models aim to generate a score for each document, with which the documents are ranked in an descending order. The goal of ranking models is to rank the more relevant documents as higher as possible. For a search task, the score is to estimate the relevancy between a query and a document [27,28,37,53,55]. For a recommendation task, the score is to estimate the relevancy between a user and a document/item [26,48,49].

2.2. Click models

Under some assumptions about user behavior, we can estimate the parameters of the click model from data via generative maximum likelihood and then use the learned click model for learning to rank. For example, the user browsing model [20] allowed users skipping some results, which extended position based model [41] to condition the examination on a previously clicked position, in addition to the position of the current result. The work [19] designed a Cascade model to separate click bias from relevance signals by assuming that users read search result pages sequentially from top to bottom. To further handle multiple clicks, researchers constructed dynamic Bayesian network model [9] and click chain model [22] by assuming a sequential user behavior over the result list. The click models are usually optimized for the likelihood of observed clicks but not the ranking performance of the overall system.

2.3. Unbiased learning algorithms

Unbiased learning algorithms [3,5,46] was proposed to address bias by leveraging counterfactual techniques like inverse propensity weighting (IPW). Different techniques have been proposed to estimate the propensity. For example, in [32], the authors introduced to shuffle position 1 and k and [47] proposed to randomly flip adjacent positions to obtain the propensity based on [46]. To infer bias without relying on randomization, [47] proposed a regression-based Expectation–Maximization (EM) algorithm which fits the Position-Based Model (PBM) into the regular click logs for personal search. [3] proposed random pairs harvesting from logs of multiple rankers in operational systems. Different from two-step methods, i.e., estimating click propensities and using them to train unbiased models, there is an another line of work. [25] introduced unbiased LambdaMART that can simultaneously conduct debiasing of click data and training of a ranker using a pairwise loss function. [5] proposed dual learning algorithm to jointly learn propensity models and ranking models from regular click data by optimizing two objective functions. TrustPBM [2] was proposed to model trust bias in the unbiased learning-to-rank setting for personal search data and [6] introduced to maximize a derived likelihood function for estimating click propensities in e-commerce search. Recently, a recurrent survival ranking framework was proposed by [29] to formulate the unbiased learning-to-rank task as to estimate the probability distribution of user’s conditional click rate. In [45], propensity ratio scoring was derived that takes a holistic treatment on both clicks and non-clicks. The work [38] formalized the problem of selection bias in learning-to-rank systems and proposed an approach for correcting for selection bias.

2.4. Bias in recommender system

The work [11] showed how understanding decision bias can improve recommender systems. There are some studies on position bias in recommender systems [16,24,35]. Another types of biases have been proposed, such as selection bias [42], algorithmic confounding bias [8], popularity bias [1,50], and marketing bias [44]. Recently, a novel attribute-based propensity estimation framework for unbiased learning in recommender systems was proposed in [40].

3. Comprehensive study of bias

In this section, we conduct a comprehensive study of bias from the following aspects. In Section 3.1, we formally give a definition of bias and discuss which factors should be considered as biases in learning to rank problem. In Section 3.2, we identify the types of biases based on the common characteristics of various biases. In Section 3.3, we then introduce some concrete biases in e-commerce feeds recommendation as examples. In Section 3.4, we summarize the bias estimation approaches and introduce how to debias in learning ranking model.

3.1. Definition of bias

In this work we focus on learning an unbiased ranking model in the ranking stage of recommender system [17]. Ranking model usually learns a scoring function from user feedback. The click data are user-centric and cheap, but it is biased, i.e., it cannot always convey the true relevance label. This will result in inconsistency between the actual ranking loss and the empirical ranking loss using click data. Thus, we need to estimate the true relevance from the user's click behavior, which will be influenced by two types of factors:

- Factors that are relatively easy to obtain during online service, such as the characteristics of user, product, and interactions between user and product. For example, user characteristics can be the user's preference for particular brand or shop, etc. We can use the price, click quantity, sales volume, rating, and comment to depict the product. These factors can be timely used as ranking features for online model.
- Factors that have an impact on user's behavior, but cannot be obtained timely or its acquisition will affect the effectiveness of online prediction. For example, a user is more likely to click the higher ranked products than the lower ones regardless of the product's relevancy. Nevertheless, we can't get the position of a product until the ranking stage and re-ranking stage (e.g. diversity-aware reordering) [39] in real-world recommender systems are finished.

Researchers have studied the influence of bias in learning to rank [5,32,47]. However, most of them only focus on one type of bias and pay little attention to different types of bias from a unified perspective. In addition, the unified bias is dependent on the specific system, i.e., a bias of one system may become a ranking feature in another system since it may be available during online inference. We argue that the bias definition should be more general and system dependent. Motivated by this issue, we aim to study bias factors from a unified point of view, and identify the appropriate bias factors in real-world recommender system. Therefore, we formally propose a bias definition as follows.

Definition 1. Bias refers to factors those will affect the customer's click behaviors, which can be available during offline training, but cannot be obtained or is inconvenient to be obtained during online inference.

3.2. Types of bias

We give three unified types of bias factors as follows.

- *Position Bias Factor* represents the location where the item is placed in, which can influence users' choice [4, 5,47]. In an e-commerce app, it can be represented as ranking order number, page, and in the left or right side of the page. Take ranking order number as example, a user is more likely to click the higher ranked products than the lower ones regardless of the product's relevancy. Only after all items are sorted by scores, the position value of an item can be obtained. Hence, we can obtain the position values for offline training, but it's missing for online inference.
- *Items' Context Bias Factor* refers to the contextual situation of item which will influence users' choice. For instance, the contextual situation could be whether there are similar items around, whether they are clicked or not, whether they are advertising products or recommended products, whether they are product or other materials (e.g., article, video), and the number of times the item was previously impressed. If there are similar items nearby (e.g., both iPhone and Samsung belong to the mobile phone category), a user's attention on Samsung may be distracted by iPhone. In [43], the authors study context of the composition of the choice set. Only after all items are sorted and exposed to the users, the contexts of target items can be observed.
- *Users' Status Bias Factor* indicates the status of user (e.g. the user's current mood) which may influence users' click behaviors. For example, whether the user have seen the products for many times may lead to different feedbacks. In [10], they assume that a user's examination action is only affected by her actions on previous results in the current query. The current status of users is inconvenient to obtain for online inference, so we incorporate it as a bias.

3.3. Example biases in real-world recommender systems

In this section, we investigate instances of biases in real-world recommender systems: position and page for the position bias factor, near exposures for the items' context bias factor, and previous exposures for the users' status bias factor.

3.3.1. Position

Position is a representation of an item's absolute ranking order in the results. In mobile feeds recommendation, four items are usually displayed in one phone screen at the same time. The position value of item is increased from left to right, and from top to bottom accordingly. The lower the position is, the less likely the user will see it [23], so that the product has less probability of being clicked.

3.3.2. Page

Page is a representation of an item's relative location. Every ten items have the same page number, and the page number of the next ten items will be increased by one. Similarly, as the page increases, the click-through rate (CTR) shows a downward trend. A commonly used practice for dealing with position or page is to inject it as an input feature in model training and then removing the bias through setting position feature to an fixed value such as missing value at serving, which leads to sub-optimal online performance [34]. We use embedding vectors instead in this paper.

3.3.3. Near exposures

Near exposures belongs to the category of items' context bias factor. Usually, several products are displayed on one phone screen at the same time in a e-commerce site. Inspired from [13,43], the near exposures will affect the CTR of target product. We also leverage the embeddings of near exposed items to represent the near exposures bias, which have good generalization ability and can fully express the item information.

3.3.4. Previous exposures

For users' status bias factor, we introduce previous exposures as an bias example. We use previous exposures and their labels (e.g., click or non-click, and order or non-order) to model user's current status. Besides, we can record their browsing time and use it in learning to rank.

3.4. Bias estimation

As mentioned above, the biases influence user's click behavior. If the click-through rate drops, the conversion rate will be decreased, which results in a lower revenue and further lead to poor recommendation. In most prior works for the search, the estimation of propensity is based on the position-based model, where the examination bias in it is the propensity needed. There are some techniques to estimate the examination bias, such as result randomization [32,46], and regression-based EM technique [47]. Nevertheless, there are some limitations of EM method. On one hand, we have to define a reasonable click model when encountering a new bias factor, then need to derive the formulas of parameter estimation. On the other hand, when the bias cannot be enumerated, EM becomes useless. As we know, conducting real-world experiments with random assignments in e-commerce site is not a good way. In this work, by leveraging neural network to deal with biases [23,56], we have the benefit of learning bias without resorting to random experiments and trying different bias combination. Furthermore, neural network can handle various representations of bias, such as numerical values, embeddings, or other substitutions.

3.5. Debiasing for learning to rank

Click-through rate prediction is an essential task in industrial applications, such as e-commerce recommendation and online advertising. However, the CTR prediction is known to suffer from the problem of biases in click data. Therefore, we need to deal with the bias for CTR prediction. Recently, deep learning based models have been proposed for CTR prediction task [17,57]. Therefore, we adopt a framework of two deep neural networks to debiasing for CTR prediction. Specifically, one neural network is used to estimate the relevance (relevance NN), i.e., we directly construct the ranking function with neural network. Another neural network is used to model biases (bias

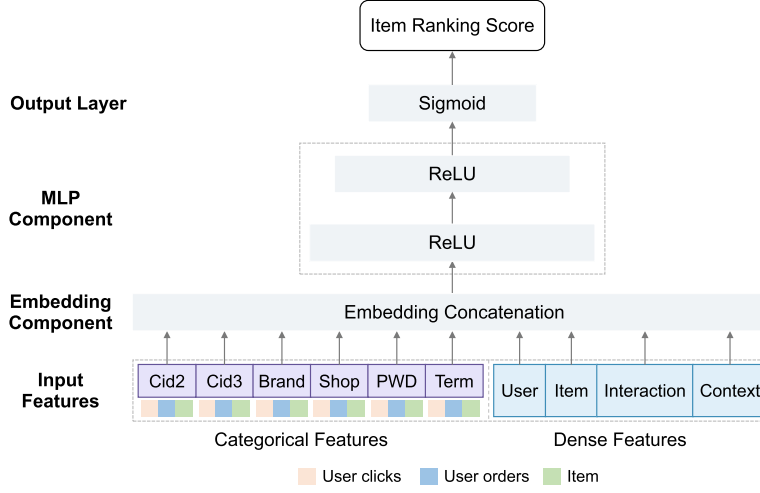


Fig. 1. Framework of the biased model.

NN). In offline training, we model different biases in the training data and give a scalar output, which serves as a bias term to the final prediction of the relevance model. Then we carry out online inference without using the bias NN.

4. Our model design experience: An unbiased neural network model

In this section, we share our experiences on designing and optimizing models to improve product recommendation with bias. We first describe the biased model and then introduce unbiased neural network models, including a prior unbiased method and our proposed unbiased models.

4.1. Biased model

In this paper, we employ a deep neural network [14,57] for CTR prediction. The network architecture is shown in Fig. 1 which we refer as a biased model. The input layer consists of dense features and categorical features. We set up a lookup layer to transform the one-hot representations of users and items into low-dimensional dense vectors, called embeddings. We concatenate the embeddings with dense features. Then, they go through two fully connected layers (Multi-layer Perceptron, i.e. MLP) with Rectified Linear Units (ReLU) activation function. The output layer has a Sigmoid activation function to produce an output score, which indicates its likelihood of belonging to the clicked category. Let a binary label $c \in \{0, 1\}$ indicate whether the click event occurs. Given the observed click label c and the output score, we minimize the widely-used cross-entropy loss function.

4.2. Unbiased model

4.2.1. Overview

We first introduce two design principles. (1) On one hand, we use the estimated bias as a weight to loss function, and then minimize the unbiased loss function. Given a pair of user request u and item x , we denote them as a feature vector $\phi(u, x)$ or ϕ as a shorthand. In learning-to-rank, the ranker f is learned with labelled data traditionally. Let y^{real} represent the true relevance label and y^{clk} denote the click label. Considering a point-wise loss $L(y^{\text{real}}, f(\phi))$,

we use the following unbiased estimator¹

$$\frac{L(y^{\text{clk}}(u, x_i), f(\phi(u, x_i)))}{p(u, x_i)},$$

where $p(u, x_i)$ is the propensity to be estimated. Specifically, the unbiased version of cross-entropy loss becomes

$$\frac{-c \log f(\phi(u, x_i)) - (1 - c) \log (1 - f(\phi(u, x_i)))}{p(u, x_i)}. \quad (1)$$

This is consistent with the empirical method used in [12], i.e., weighting each data-point with their corresponding propensity. (2) On the other hand, we use the estimated bias in a similar way to the position-based model [7], the difference lies in that our bias is not the examination bias only based on position. Now we are going to utilize these two design principles and introduce each model in detail.

4.2.2. Regression EM and relevance NN

In this section, we first introduce an unbiased baseline model, which fits into the first design principle. We employ the regression-based Expectation–Maximization method to estimate the position bias [47] and combine it with a relevance neural network, which serve as the score function $f(\phi)$. Following [47], we use gradient boosting decision tree (GBDT) in the regression-based EM method. Denote the position bias estimated by regression-based EM by p^{em} . For Equation (1), we estimate $p(u, x_i)$ by p^{em} and use the relevance probability p^{rel} to replace $f(\phi)$:

$$\frac{-c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})}{p^{\text{em}}}. \quad (2)$$

According to each item’s position, we can use the weight $\frac{1}{p^{\text{em}}}$ for all of the samples or only for the clicked samples.

4.2.3. Bias NN and relevance NN

From now on, we introduce our unbiased model. As shown in Fig. 2, the proposed model consists of two components: (a) modeling bias with a MLP from bias embeddings; (b) learning the relevance prediction with another MLP from dense and categorical features. We give four models as follows: Bias NN as Weight, Multiplication Combination, Addition Combination, and Combined Loss.

Bias NN as weight. This models is applied with the first design principle. For Equation (1), we estimate $p(u, x_i)$ by the bias NN and use the relevance probability p^{rel} to replace $f(\phi)$. Let p^{bias} denote the position bias estimated by bias NN. Then, the unbiased cross-entropy loss for each training data becomes

$$\frac{-c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})}{p^{\text{bias}}}. \quad (3)$$

Similarly, we can apply the weight $\frac{1}{p^{\text{bias}}}$ to all the samples or only to the clicked samples.

Please note that, in the subsequent sections, we leverage the second design principle.

Multiplication combination. In this section, we multiply the relevance probability p^{rel} with bias probability p^{bias} . Take one training sample as an example, the cross-entropy loss is

$$-c \log p^{\text{clk}} - (1 - c) \log (1 - p^{\text{clk}}), \quad (4)$$

where p^{clk} denotes the multiplication $p^{\text{rel}} \times p^{\text{bias}}$.

¹Please refer to the supplementary material for derivations.

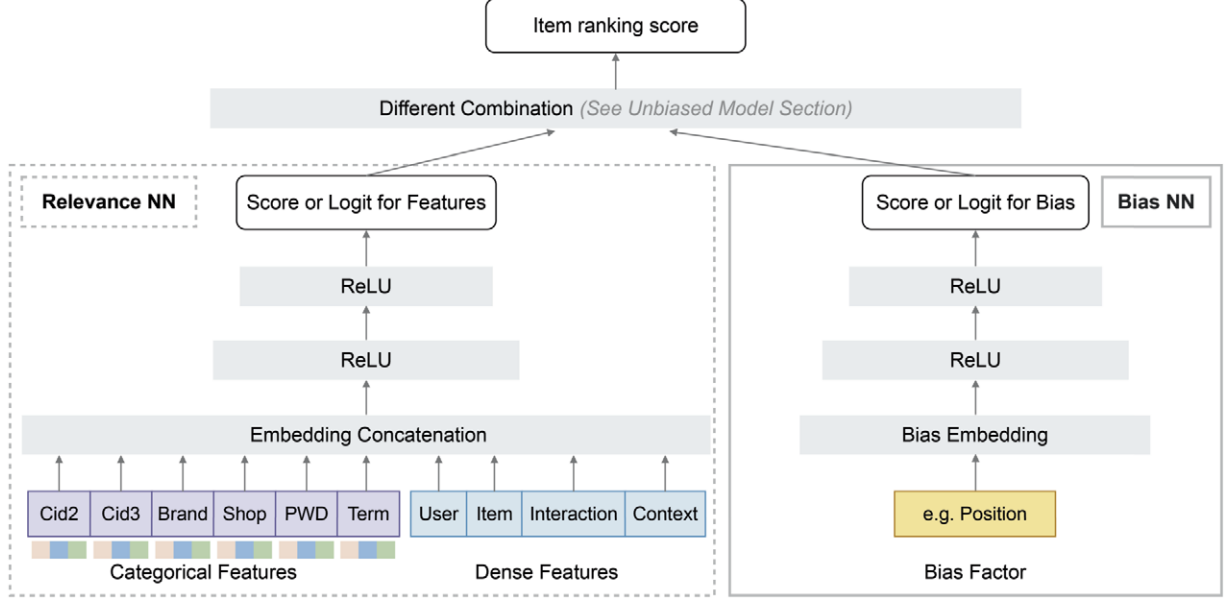


Fig. 2. Framework of the proposed unbiased model.

Addition combination. Different from the above multiplication combination, we add the relevance logit $\text{logit}^{\text{rel}}$ with bias logit $\text{logit}^{\text{bias}}$ [56] and then use Sigmoid function σ . That is

$$-c \log \sigma(\text{logit}^{\text{clk}}) - (1 - c) \log(1 - \sigma(\text{logit}^{\text{clk}})), \quad (5)$$

where $\text{logit}^{\text{clk}}$ denotes the sum $\text{logit}^{\text{rel}} + \text{logit}^{\text{bias}}$.

Combined loss. In previous sections, we consider optimizing a single cross-entropy loss. Inspired by [36], we define the following combined loss, consisting of two cross-entropy losses from the final output CTR and from the relevance probability, respectively. It not only adopt the click signals as the proxy of the relevance but also use them for click prediction. The combined loss is

$$-c \log p^t - (1 - c) \log(1 - p^t) - c \log p^{\text{rel}} - (1 - c) \log(1 - p^{\text{rel}}),$$

where p^t can be represented by $\sigma(\text{logit}^{\text{clk}})$ or p^{clk} .

At last, we give a summary of models in Table 1, including the biased neural network, regression EM and relevance neural network, and our proposed unbiased models. The notations will be used in the experiment section.

5. Experiments

In this section, we first introduce the experimental settings. Then we conduct extensive experiments to evaluate the effectiveness of the proposed unbiased models.

5.1. Setup

Our experiments are based on offline evaluation and online experiments. For offline evaluation, we use the standard framework for supervised learning to rank evaluation by splitting the data into training, validation, and test datasets. We trained a 2-layer MLP model on the training data set. For the relevance NN, the hidden units of the two fully-connected layers are 256 and 128, respectively. For the bias NN, the hidden units are 16 and 8, respectively.

Table 1
A summary of models

Model	Notation	Loss function
<i>Biased Baseline</i>		
Biased Model	Biased NN	$-c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})$
<i>Unbiased Baseline</i>		
Regression EM and Relevance NN	Reg-EM&Rel NN	$\frac{-c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})}{p^{\text{em}}}$
<i>Proposed Unbiased Model</i>		
Bias NN as Weight	As Weight	$\frac{-c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})}{p^{\text{bias}}}$
Multiplication Combination	Multiplication	$-c \log p^{\text{clk}} - (1 - c) \log (1 - p^{\text{clk}})$
Addition Combination	Addition	$-c \log \sigma(\text{logit}^{\text{clk}}) - (1 - c) \log (1 - \sigma(\text{logit}^{\text{clk}}))$
Combined Loss	Combined Loss	$-c \log p^t - (1 - c) \log (1 - p^t) - c \log p^{\text{rel}} - (1 - c) \log (1 - p^{\text{rel}})$

Table 2
Statistics of the datasets used in offline evaluation

Label	#Train	#Test
Impressions	622,596,211	98,732,799
Clicks	43,876,602	6,477,409
Orders	1,434,837	234,463
Total	667,907,650	105,444,671

We performed Adam optimizer [33] in training with a mini-batch size of 2048 and used an initial learning rate 0.001. The experiments were conducted on a 4 cards GPU and is built on TensorFlow.² In addition, we evaluate our proposed unbiased models through online A/B testing.

5.2. Datasets

To evaluate the proposed approach, we collect traffic logs, including item impressions, clicks and orders label, from a real world e-commerce portal. The input features consist of dense features and categorical features, which describe different aspect of an data sample. For example, the dense features including the features describing user profile, item profile, and user-item interactions. Table 2 summarizes basic statistics of the used dataset in offline evaluation. The training data set consists of about 0.66 billions samples, where each sample has an observed click label 0 or 1. For offline test evaluation, we design two datasets: *click dataset* and *order dataset*. On the click dataset, the impression labels are recorded as 0, the click and order labels are recorded as 1. While on the order dataset, the impression and click labels are recorded as 0, and the order labels are recorded as 1.

5.3. Models for comparison

Biased NN and **Reg-EM&Rel NN** serve as our biased and unbiased baselines, respectively.

- **Biased NN**: The biased baseline. In this paper, Biased NN is instantiated as a common used fully-connected deep neural network for CTR prediction. Actually, we can use models such as Wide&Deep [14], DIN [57], and other baselines as alternatives.
- **Reg-EM&Rel NN**: The unbiased baseline. The relevance probability (of all samples or only positive clicked samples) is weighted by their corresponding position propensity from regression EM. More detailed, Reg-EM&Rel-NN utilizes the above mentioned fully-connected deep neural network for CTR prediction and weights each data point in the cross-entropy loss with their corresponding position bias estimated by a regression-based Expectation Maximization method, which is inspired from [12] and [47].

²<https://www.tensorflow.org/>

Table 3
Comparison of the item ranking performance of each model on order dataset

Bias	Model	Prec@2	Prec@4	Prec@12	MRR@2	MRR@4	MRR@12
–	Biased NN	0.1656	0.1502	0.1267	0.2307	0.2863	0.3245
Position	Reg-EM&Rel NN	0.1650	0.1496	0.1266	0.2289	0.2842	0.3226
Near exposures	As Weight	0.1666	0.1502	0.1267	0.2318	0.2868	0.3250
	Multiplication	0.1673	0.1506	0.1268	0.2330	0.2881	0.3263
	Addition	0.1672	0.1508	0.1266	0.2321	0.2875	0.3252
	Combined loss	0.1662	0.1503	0.1269	0.2311	0.2865	0.3250
Position	As Weight	0.1661	0.1500	0.1267	0.2311	0.2862	0.3246
	Multiplication	0.1596	0.1475	0.1262	0.2201	0.2763	0.3152
	Addition	0.1625	0.1484	0.1264	0.2251	0.2804	0.3191
	Combined loss	0.1660	0.1502	0.1267	0.2306	0.2860	0.3241
Page	As Weight	0.1666	0.1500	0.1267	0.2318	0.2867	0.3251
	Multiplication	0.1637	0.1485	0.1264	0.2272	0.2819	0.3207
	Addition	0.1651	0.1494	0.1263	0.2295	0.2845	0.3227
	Combined loss	0.1670	0.1500	0.1268	0.2327	0.2872	0.3258

- **Bias NN&Rel NN**: Model using bias NN and relevance NN. Specifically, Bias NN&Rel NN uses the deep neural network for CTR prediction and incorporates bias as a separate model by following [10] and [23]. We also refer to [10] and [56] in terms of the combination ways for these two networks.

5.4. Evaluation metrics

We describe metrics used in offline evaluation and online experiments. The offline comparisons are made on two tasks [10]: item ranking and click prediction.

- For item ranking task, we use the scores of relevance neural network to ranking the items. We adopt *Precision (Prec)@k* and *Mean Reciprocal Rank (MRR)@k* [18]. Specifically, for *Prec@k*, we set a rank threshold k , compute the relevant percentage in top- k items, and ignore items ranked lower than k . Reciprocal rank accounts for the first relevant item in top- k items and *MRR@k* is the mean Reciprocal rank across multiple requests.
- For click prediction task, we ranking all the items with predicted CTR. We use *Area Under the ROC Curve (AUC)* [21,57] as evaluation metrics for the goodness of predicting user’s clicks.
- More importantly, we conduct A/B testing to further validate our unbiased model. We use three important online indicators: *Click-through Rate (CTR)*, *Conversion Rate (CVR)*, and *Gross Merchandise Volume (GMV)* to measure the online performance.

Note that all metrics are the higher the better.

5.5. Result from model comparison

For item ranking, we report the results on order dataset in Table 3. For the sake of length limitation, we put the result of item ranking on click dataset in the supplementary material. The first column in Table 3 denotes different biases and the second column is the compared models. We compare the unbiased models with biased model and all the values in bold means it is better than the corresponding results of the biased NN. We can observe:

- Overall, the unbiased model using near exposures achieve better performance than the biased NN. On the order dataset, the bias near exposures stand out and its exploitation by using multiplication has the best metrics. While on the click dataset, the exploitation of near exposures by using addition performs best.
- For position bias, it is worse than the biased NN. As for page bias, the combined loss almost outperforms biased NN on the order dataset. We suspect that, since it is an indirect optimization of relevance NN for the unbiased NN, so the item ranking performance is likely to perform weaker than the biased NN (an direct optimization of

Table 4
Comparison of the click prediction performance on order dataset

Bias	Model	AUC
–	Biased NN	0.7335
Position	Reg-EM&Rel NN	0.7323
Near exposures	As Weight	0.7315
	Multiplication	0.7323
	Addition	0.7331
	Combined loss	0.7308
Position	As Weight	0.7329
	Multiplication	0.7390
	Addition	0.7429
	Combined loss	0.7393
Page	As Weight	0.7328
	Multiplication	0.7378
	Addition	0.7398
	Combined loss	0.7378

Table 5
Online A/B testing results. “*” indicates a statistically significant improvements ($p < 0.01$) over the baseline

Model	CTR	CVR	GMV
Biased NN (Base)	+0.0%	+0.0%	+0.0%
Near exposures (Addition)	+2.4%*	+1.2%*	+6.5%*

relevance NN). On the other hand, it may result from that the granularity of position and page in e-commerce site is too fine for users to perceive compared with near exposures. When users browse and click products, they will first be affected by the items around the target items. Secondly, they will be influenced by the depth of pages.

- Besides, biased NN performs better than Reg-EM&Rel NN. We guess it may because the propensity result of regression-based EM method is dependent on training data, which lead to poorer performance than the biased model.

The above observations show that, by using neural network and bias embedding to model the near exposures bias, we can achieve better performances of item ranking than the baselines both in the click and order datasets.

For the click prediction task, we compare the click prediction performance of biased model and unbiased models on the order dataset, as shown in Table 4. All the lines in bold means better performance than the biased model. We have the following observations. Biased NN model outperforms Reg-EM&Rel NN. Different from the item ranking results in Table 3, the position or page bias have better performances than the near exposures. Overall, modelling position bias outperforms using page bias. We can observe that the exploitation of position by using addition combination has the highest AUC value. The results show that, by using neural network and bias embedding to model the position or page bias, we can improve the performance of click prediction on the order dataset.

5.6. Result from online A/B testing

We evaluate the performances of **Biased NN** and the unbiased model **Near exposures (Addition)** (i.e., exploiting near exposures using addition combination, which has the best overall performance concerning item ranking performance on the click and order datasets) by deploying it at a real-world e-commerce portal. The online A/B testing was carried out for one month. We report the real-world online experimental results in Table 5. Compared with Biased NN, we can see that the unbiased model contributes up to 2.4%, 1.2% and 6.5% in CTR, CVR and GMV, respectively. This results further proves the effectiveness of our unbiased model.

6. Conclusion and future work

In consideration of there has no unified bias analysis, we first perform a comprehensive study of biases to identify the appropriate bias factors in real-world learning to rank systems, and discuss the approaches to remove the effect of biases. We then share our experiences derived from designing and optimizing models to improve feeds recommendation with bias. To uncover the effects of biases and achieve better ranking performance, we propose several unbiased models by leveraging deep neural network. Empirically, we conduct extensive offline experiments on datasets gathered from a real-world e-commerce portal and validate the effectiveness of our method by performing online A/B testing in a real-world e-commerce recommender system. Experimental results demonstrate that our unbiased models is able to achieve superior performance against state-of-the-art baselines. In this paper, we have given an identification of three types of bias factors, while it is possible that more bias factors exist. Therefore, one possible future work is to investigate more types of potential biases in ranking systems. In our experiments, we consider one bias as an input. However, there are some cases with two or more biases simultaneously. It would be interesting to study how to debias multiple biases at the same time, which can also be our future work.

Acknowledgements

We thank the reviewers and associate editor. This research is supported by the research grant from Natural Sciences and Engineering Research Council of Canada (NSERC).

Appendix A. Derivations of unbiased point-wise loss

One challenge of industrial recommendation systems is the scalability. To achieve a trade-off between model quality and efficiency, a popular choice is to use deep neural network-based point-wise ranking models. Given a pair of user request u and item x , we denote them as a feature vector $\phi(u, x)$. Without ambiguity, we use ϕ as a shorthand. In learning-to-rank, the ranker f is learned with labeled data traditionally. Let I_u denote the set of items with respect to u . Let y^{real} represent the true relevance label. Considering a point-wise loss $L(y^{\text{real}}, f(\phi))$, the risk function is defined as

$$R^{\text{real}}(f) = \int L(y^{\text{real}}, f(\phi)) d\Pr(y^{\text{real}}, \phi), \quad (6)$$

where $\Pr(y^{\text{real}}, \phi)$ denotes the probability distribution on y^{real} and ϕ . However, we usually do not have the true relevance label and use the click label as an alternative. Let y^{click} be the click label and the risk function becomes

$$R^{\text{click}}(f) = \int L(y^{\text{click}}, f(\phi)) d\Pr(y^{\text{click}}, \phi). \quad (7)$$

Following [25], we assume that there exists a $p(u, x_i)$ satisfying $\Pr(y^{\text{click}}|\phi) = p(u, x_i) \Pr(y^{\text{real}}|\phi)$. Thus, we have

$$\begin{aligned} R^{\text{unbiased}}(f) &= \int \frac{L(y^{\text{click}}, f(\phi))}{p(u, x_i)} d\Pr(y^{\text{click}}, \phi) \\ &= \int \frac{L(y^{\text{click}}, f(\phi))}{\frac{\Pr(y^{\text{click}}|\phi)}{\Pr(y^{\text{real}}|\phi)}} d\Pr(y^{\text{click}}, \phi) \\ &= \int L(y^{\text{click}}, f(\phi)) d\Pr(y^{\text{real}}, \phi) \\ &= \int L(y^{\text{real}}, f(\phi)) d\Pr(y^{\text{real}}, \phi) = R^{\text{real}}(f). \end{aligned}$$

Table 6
Comparison of the item ranking performance of each model on click dataset

Bias	Competitor	Prec@2	Prec@4	Prec@12	MRR@2	MRR@4	MRR@12
/	Biased NN	0.3235	0.2844	0.2305	0.3983	0.4500	0.4830
Position	Reg-EM&Rel NN	0.3216	0.2828	0.2301	0.3955	0.4468	0.4804
Near exposures	As Weight	0.3241	0.2841	0.2304	0.3987	0.4500	0.4831
	Multiplication	0.3239	0.2843	0.2305	0.3984	0.4501	0.4831
	Addition	0.3261	0.2849	0.2302	0.4011	0.4525	0.4852
	Combined Loss	0.3229	0.2832	0.2305	0.3975	0.4489	0.4823
Position	As Weight	0.3230	0.2841	0.2304	0.3975	0.4492	0.4825
	Multiplication	0.3036	0.2735	0.2281	0.3707	0.4239	0.4586
	Addition	0.3106	0.2766	0.2286	0.3807	0.4329	0.4672
	Combined Loss	0.3227	0.2838	0.2302	0.3970	0.4484	0.4816
Page	As Weight	0.3244	0.2838	0.2304	0.3991	0.4503	0.4835
	Multiplication	0.3115	0.2766	0.2287	0.3820	0.4340	0.4684
	Addition	0.3166	0.2790	0.2283	0.3888	0.4402	0.4736
	Combined Loss	0.3237	0.2838	0.2304	0.3986	0.4497	0.4829

Now we can obtain an unbiased ranker $\hat{f}^{\text{unbiased}}$, i.e.,

$$\hat{f}^{\text{unbiased}} = \arg \min_f \sum_u \sum_i \frac{L(y^{\text{click}}(u, x_i), f(\phi(u, x_i)))}{p(u, x_i)}.$$

Appendix B. Experimental results for the click dataset

We show the experimental results for the click dataset in Table 6. On the click dataset, the exploitation of *near exposures* by using addition performs best.

References

- [1] H. Abdollahpour, R. Burke and B. Mobasher, Controlling popularity bias in learning-to-rank recommendation, in: *RecSys*, ACM, 2017, pp. 42–46.
- [2] A. Agarwal, X. Wang, C. Li, M. Bendersky and M. Najork, Addressing trust bias for unbiased learning-to-rank, in: *WWW*, ACM, 2019, pp. 4–14.
- [3] A. Agarwal, I. Zaitsev and T. Joachims, Consistent position bias estimation without online interventions for learning-to-rank, 2018, CoRR [arXiv:1806.03555](https://arxiv.org/abs/1806.03555).
- [4] A. Agarwal, I. Zaitsev, X. Wang, C. Li, M. Najork and T. Joachims, Estimating position bias without intrusive interventions, in: *WSDM*, ACM, 2019, pp. 474–482.
- [5] Q. Ai, K. Bi, C. Luo, J. Guo and W. Bruce Croft, Unbiased learning to rank with unbiased propensity estimation, *SIGIR* (2018), 385–394.
- [6] G. Aslanyan and U. Porwal, Position bias estimation for unbiased learning-to-rank in ecommerce search, in: *SPIRE*, 2019, pp. 47–64.
- [7] A. Borisov, I. Markov, M. de Rijke and P. Serdyukov, A neural click model for web search, in: *WWW*, ACM, 2016, pp. 531–541.
- [8] A.J.B. Chaney, B.M. Stewart and B.E. Engelhardt, How algorithmic confounding in recommendation systems increases homogeneity and decreases utility, in: *RecSys*, ACM, 2018, pp. 224–232.
- [9] O. Chapelle and Y. Zhang, A dynamic Bayesian network click model for web search ranking, in: *WWW*, ACM, 2009, pp. 1–10.
- [10] J. Chen, J. Mao, Y. Liu, M. Zhang and S. Ma, A context-aware click model for web search, in: *WSDM*, ACM, 2020, pp. 88–96. doi:[10.1145/3336191.3371819](https://doi.org/10.1145/3336191.3371819).
- [11] L. Chen, M. de Gemmis, A. Felfernig, P. Lops, F. Ricci and G. Semeraro, Human decision making and recommender systems, *ACM Trans. Interact. Intell. Syst.* **3**(3) (2013), 17:1–17:7.
- [12] R.-C. Chen, Q. Ai, G. Jayasinghe and W. Bruce Croft, Correcting for recency bias in job recommendation, in: *CIKM*, ACM, 2019, pp. 2185–2188.
- [13] S. Chen and T. Joachims, Predicting matchups and preferences in context, in: *SIGKDD*, ACM, 2016, pp. 775–784.

- [14] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu and H. Shah, Wide & deep learning for recommender systems, in: *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, 2016, pp. 7–10. doi:[10.1145/2988450.2988454](https://doi.org/10.1145/2988450.2988454).
- [15] X. Chu, J. Zhao, L. Zou and D. Yin, H-ernie: A multi-granularity pre-trained language model for web search, in: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022, pp. 1478–1489. doi:[10.1145/3477495.3531986](https://doi.org/10.1145/3477495.3531986).
- [16] A. Collins, D. Tkaczyk, A. Aizawa and J. Beel, Position bias in recommender systems for digital libraries, in: *International Conference on Information*, Springer, 2018, pp. 335–344.
- [17] P. Covington, J. Adams and E. Sargin, Deep neural networks for youtube recommendations, in: *RecSys*, ACM, 2016, pp. 191–198. doi:[10.1145/2959100.2959190](https://doi.org/10.1145/2959100.2959190).
- [18] N. Craswell, Mean reciprocal rank, in: *Encyclopedia of Database Systems*, 2nd edn, Springer, 2018.
- [19] N. Craswell, O. Zoeter, M. Taylor and B. Ramsey, An experimental comparison of click position-bias models, in: *WSDM*, ACM, 2008, pp. 87–94. doi:[10.1145/1341531.1341545](https://doi.org/10.1145/1341531.1341545).
- [20] G. Dupret and B. Piwowarski, A user browsing model to predict search engine click data from past observations, in: *SIGIR*, ACM, 2008, pp. 331–338.
- [21] T. Fawcett, An introduction to ROC analysis, *Pattern Recognit. Lett.* **27**(8) (2006), 861–874. doi:[10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010).
- [22] F. Guo, C. Liu, A. Kannan, T. Minka, M.J. Taylor, Y.M. Wang and C. Faloutsos, Click chain model in web search, in: *WWW*, ACM, 2009, pp. 11–20.
- [23] H. Guo, J. Yu, Q. Liu, R. Tang and Y. Zhang, PAL: A position-bias aware learning framework for CTR prediction in live recommender systems, in: *RecSys*, ACM, 2019, pp. 452–456.
- [24] K. Hofmann, A. Schuth, A. Bellogin and M. De Rijke, Effects of position bias on click-based recommender evaluation, in: *ECIR*, Springer, 2014, pp. 624–630.
- [25] Z. Hu, Y. Wang, Q. Peng and H. Li, Unbiased lambdamart: An unbiased pairwise learning-to-rank algorithm, in: *WWW*, ACM, 2019, pp. 2830–2836.
- [26] C. Huang, J. Chen, L. Xia, Y. Xu, P. Dai, Y. Chen, L. Bo, J. Zhao and J.X. Huang, Graph-enhanced multi-task learning of multi-level transition dynamics for session-based recommendation, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, 2021, pp. 4123–4130.
- [27] X. Huang and Q. Hu, A Bayesian learning approach to promoting diversity in ranking for biomedical information retrieval, in: *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR 2009, Boston, MA, USA, July 19–23, 2009, J. Allan, J.A. Aslam, M. Sanderson, C. Zhai and J. Zobel, eds, ACM, 2009 pp. 307–314.
- [28] X. Huang, F. Peng, D. Schuurmans, N. Cercone and S.E. Robertson, Applying machine learning to text segmentation for information retrieval, *Inf. Retr.* **6**(3–4) (2003), 333–362. doi:[10.1023/A:1026028229881](https://doi.org/10.1023/A:1026028229881).
- [29] J. Jin, Y. Fang, W. Zhang, K. Ren, G. Zhou, J. Xu, Y. Yu, J. Wang, X. Zhu and K. Gai, A deep recurrent survival model for unbiased ranking, in: *SIGIR*, ACM, 2020, pp. 29–38.
- [30] T. Joachims, L.A. Granka, B. Pan, H. Hembrooke and G. Gay, Accurately interpreting clickthrough data as implicit feedback, in: *SIGIR*, Vol. 5, ACM, 2005, pp. 154–161. doi:[10.1145/1076034.1076063](https://doi.org/10.1145/1076034.1076063).
- [31] T. Joachims, L.A. Granka, B. Pan, H. Hembrooke, F. Radlinski and G. Gay, Evaluating the accuracy of implicit feedback from clicks and query reformulations in web search, *ACM Trans. Inf. Syst.* **25**(2) (2007), 7. doi:[10.1145/1229179.1229181](https://doi.org/10.1145/1229179.1229181).
- [32] T. Joachims, A. Swaminathan and T. Schnabel, Unbiased learning-to-rank with biased feedback, in: *WSDM*, ACM, 2017, pp. 781–789.
- [33] D.P. Kingma and J. Ba, Adam: A method for stochastic optimization, in: *ICLR*, 2015.
- [34] X. Ling, W. Deng, C. Gu, H. Zhou, C. Li and F. Sun, Model ensemble for click prediction in bing search ads, in: *WWW*, ACM, 2017, pp. 689–698.
- [35] Z. Lu, Z. Dou, J. Lian, X. Xie and Q. Yang, Content-based collaborative filtering for news topic recommendation, in: *AAAI*, 2015, pp. 217–223.
- [36] X. Ma, L. Zhao, G. Huang, Z. Wang, Z. Hu, X. Zhu and K. Gai, Entire space multi-task model: An effective approach for estimating post-click conversion rate, in: *SIGIR*, ACM, 2018, pp. 1137–1140.
- [37] J. Miao, J.X. Huang and J. Zhao, TopPRF: A probabilistic framework for integrating topic space into pseudo relevance feedback, *ACM Transactions on Information Systems (TOIS)* **34**(4) (2016), 1–36. doi:[10.1145/2956234](https://doi.org/10.1145/2956234).
- [38] Z. Ovaysi, R. Ahsan, Y. Zhang, K. Vasilaky and E. Zheleva, Correcting for selection bias in learning-to-rank systems, in: *WWW*, ACM/IW3C2, 2020, pp. 1863–1873. doi:[10.1145/3366423.3380255](https://doi.org/10.1145/3366423.3380255).
- [39] C. Pei, Y. Zhang, Y. Zhang, F. Sun, X. Lin, H. Sun, J. Wu, P. Jiang, J. Ge, W. Ou et al., Personalized re-ranking for recommendation, in: *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019, pp. 3–11. doi:[10.1145/3298689.3347000](https://doi.org/10.1145/3298689.3347000).
- [40] Z. Qin, S.J. Chen, D. Metzler, Y. Noh, J. Qin and X. Wang, Attribute-based propensity for unbiased learning in recommender systems: Algorithm and case studies, in: *SIGKDD*, ACM, 2020.
- [41] M. Richardson, E. Dominowska and R. Ragno, Predicting clicks: Estimating the click-through rate for new ads, in: *WWW*, ACM, 2007, pp. 521–530.
- [42] T. Schnabel, A. Swaminathan, A. Singh, N. Chandak and T. Joachims, Recommendations as treatments: Debiasing learning and evaluation, in: *ICML*, 2016, pp. 1670–1679.
- [43] A. Seshadri, A. Peysakhovich and J. Ugander, Discovering context effects from raw choice data, in: *ICML*, PMLR, 2019, pp. 5660–5669.
- [44] M. Wan, J. Ni, R. Misra and J.J. McAuley, Addressing marketing bias in product recommendations, in: *WSDM*, ACM, 2020, pp. 618–626. doi:[10.1145/3336191.3371855](https://doi.org/10.1145/3336191.3371855).

- [45] N. Wang, X. Wang and H. Wang, Unbiased learning to rank via propensity ratio scoring, 2020, CoRR [arXiv:2005.08480](https://arxiv.org/abs/2005.08480).
- [46] X. Wang, M. Bendersky, D. Metzler and M. Najork, Learning to rank with selection bias in personal search, in: *SIGIR*, ACM, 2016, pp. 115–124.
- [47] X. Wang, N. Golbandi, M. Bendersky, D. Metzler and M. Najork, Position bias estimation for unbiased learning to rank in personal search, in: *WSDM*, ACM, 2018, pp. 610–618.
- [48] W. Wei, C. Huang, L. Xia, Y. Xu, J. Zhao and D. Yin, Contrastive meta learning with behavior multiplicity for recommendation, in: *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, 2022, pp. 1120–1128. doi:[10.1145/3488560.3498527](https://doi.org/10.1145/3488560.3498527).
- [49] X. Wu, H. Chen, J. Zhao, L. He, D. Yin and Y. Chang, Unbiased learning to rank in feeds recommendation, in: *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, 2021, pp. 490–498. doi:[10.1145/3437963.3441751](https://doi.org/10.1145/3437963.3441751).
- [50] L. Yang, Y. Cui, Y. Xuan, C. Wang, S.J. Belongie and D. Estrin, Unbiased offline recommender evaluation for missing-not-at-random implicit feedback, in: *RecSys*, ACM, 2018, pp. 279–287.
- [51] Y. Yue, R. Patel and H. Roehrig, Beyond position bias: Examining result attractiveness as a source of presentation bias in clickthrough data, in: *WWW*, ACM, 2010, pp. 1011–1018.
- [52] L. Zhang, L. Shi, J. Zhao, J. Yang, T. Lyu, D. Yin and H. Lu, A gnn-based multi-task learning framework for personalized video search, in: *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, 2022, pp. 1386–1394. doi:[10.1145/3488560](https://doi.org/10.1145/3488560).
- [53] J. Zhao, J.X. Huang and B. Ben He, CRTER: Using cross terms to enhance probabilistic information retrieval, in: *Proceeding of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2011*, Beijing, China, July 25–29, 2011, W.-Y. Ma, J.-Y. Nie, R. Baeza-Yates, T.-S. Chua and W. Bruce, eds, ACM, 2011, pp. 155–164.
- [54] J. Zhao, J.X. Huang, H. Deng, Y. Chang and L. Xia, Are topics interesting or not? An lda-based topic-graph probabilistic model for web search personalization, *ACM Transactions on Information Systems (TOIS)* **40**(3) (2021), 1–24. doi:[10.1145/3476106](https://doi.org/10.1145/3476106).
- [55] J. Zhao, J.X. Huang and Z. Ye, Modeling term associations for probabilistic information retrieval, *ACM Transactions on Information Systems (TOIS)* **32**(2) (2014), 1–47. doi:[10.1145/2590988](https://doi.org/10.1145/2590988).
- [56] Z. Zhao, L. Hong, L. Wei, J. Chen, A. Nath, S. Andrews, A. Kumthekar, M. Sathiamoorthy, X. Yi and E.H. Chi, Recommending what video to watch next: A multitask ranking system, in: *RecSys*, ACM, 2019, pp. 43–51.
- [57] G. Zhou, X. Zhu, C. Song, Y. Fan, H. Zhu, X. Ma, Y. Yan, J. Jin, H. Li and K. Gai, Deep interest network for click-through rate prediction, in: *SIGKDD*, ACM, 2018, pp. 1059–1068.